

MATH337 Reference

Krish A. Patel

May 4, 2026

1 Chapter 1

1.1 Traffic Intensity

$$I = \frac{La}{R}$$

where

- L is the packet size [bits/packet],
- a is the arrival rate [packets/sec],
- R is the transmission rate [bits/sec].

Note: as traffic intensity approaches 1, the queuing delay approaches infinity asymptotically.

1.2 Transmission Delay

$$\begin{aligned} \text{Node-to-node: } d_{\text{trans}} &= \frac{L}{R} \\ \text{End-to-end: } d_{\text{trans}} &= N \frac{L}{R} \end{aligned}$$

where

- L is the packet size [bits],
- N is the number of nodes,
- R is the transmission rate [bits/sec].

1.3 Propagation Delay

$$d_{\text{prop}} = \frac{d}{s}$$

where

- d is the distance [meters],
- s is the speed of light [meters/sec].

1.4 Queuing Delay

$$d_{\text{queue}} = \frac{I \cdot (L/R)}{1 - I}$$

where

- I is the traffic intensity,
- L is the packet size [bits],
- R is the transmission rate [bits/sec].

1.5 Total Nodal Delay

$$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$$

where

- d_{proc} is the processing delay,
- d_{queue} is the queuing delay,
- d_{trans} is the transmission delay,
- d_{prop} is the propagation delay.

1.6 End-to-End Throughput

$$\min \{R_1, R_2, \dots, R_N\}$$

where R_i is the transmission rate at node i . Assuming that there is only one source and one destination, the end-to-end throughput is dependent on the bottleneck link's transmission rate.

2 Chapter 2

2.1 Client-Server File Distribution

$$D_{cs} \geq \max \left\{ \frac{NF}{u_s}, \frac{F}{d_{\min}} \right\}$$
$$d_{\min} = \min \{d_1, d_2, \dots, d_N\}$$

where

- N is the number of peers,
- F is the file size [bits],
- u_s is the server upload rate [bits/sec],
- $d_{\min}$ is the minimum peer download rate [bits/sec],
- d_i is the download rate of peer i [bits/sec].

This assumes one server is used to distribute the file to multiple clients. Oftentimes, the $\geq$ can be replaced with $=$.

2.2 Peer-to-Peer File Distribution

$$D_{P2P} \geq \max \left\{ \frac{F}{u_s}, \frac{F}{d_{\min}}, \frac{NF}{u_s + \sum_{i=1}^N u_i} \right\}$$

where

- N is the number of peers,
- F is the file size [bits],
- u_s is the server upload rate [bits/sec],
- $d_{\min}$ is the minimum peer download rate [bits/sec],
- u_i is the upload rate of peer i [bits/sec].

This assumes that the server only needs to send one copy of the file to the peers, who can then redistribute it among themselves. Oftentimes, the $\geq$ can be replaced with $=$. Note: client-server distribution time scales linearly with N , while peer-to-peer distribution time scales much slower with N , always remaining under the client-server distribution time.

3 Chapter 3

3.1 Utilization Rate

$$\text{Stop-and-wait: } U_{\text{sender}} = \frac{L/R}{RTT + L/R}$$

$$\text{Pipelined: } U_{\text{sender}} = \frac{NL/R}{RTT + L/R}$$

where

- L is the packet size [bits],
- R is the transmission rate [bits/sec],
- N is the number of packets sent at once before receiving an ACK,
- RTT is the round-trip time [sec].

Measures the fraction of time the sender is busy sending data.

3.2 Estimated RTT

$$\text{EstimatedRTT} = (1 - \alpha) \cdot \text{EstimatedRTT} + \alpha \cdot \text{SampleRTT}$$

where

- α is the smoothing factor (recommended $\alpha = 0.125$),
- EstimatedRTT is the estimated round-trip time [sec],
- SampleRTT is the sample round-trip time [sec].

This uses an exponential weighted moving average (EWMA), in which the weight of a particular **SampleRTT** decreases exponentially fast with more updates.

3.3 RTT Deviation

$$\text{DevRTT} = (1 - \beta) \cdot \text{DevRTT} + \beta \cdot |\text{SampleRTT} - \text{EstimatedRTT}|$$

where

- β is the smoothing factor (recommended $\beta = 0.25$),
- DevRTT is the round-trip time deviation [sec],

- SampleRTT is the sample round-trip time [sec],
- EstimatedRTT is the estimated round-trip time [sec].

This uses an EWMA on the absolute difference between the **SampleRTT** and **EstimatedRTT**.

3.4 Timeout Interval

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \cdot \text{DevRTT}$$

where

- TimeoutInterval is in [sec] (initially 1 second),
- EstimatedRTT is the estimated round-trip time [sec],
- DevRTT is the round-trip time deviation [sec].

The timeout interval determines when to retransmit a packet if no ACK is received. After a timeout event, the timeout interval is doubled, but after successful receipt of an ACK, the timeout interval is again calculated using the formula above.

3.5 TCP Send Rate

$$\frac{\text{cwnd}}{\text{RTT}}$$

where

- cwnd is the congestion window [bytes],
- RTT is the round-trip time [sec].

3.6 Receive Window (Flow Control)

$$\begin{aligned} \text{rwnd} &= \text{RcvBuffer} - [\text{LastByteRcvd} - \text{LastByteRead}] \\ \text{RcvBuffer} &\geq \text{LastByteRcvd} - \text{LastByteRead} \end{aligned}$$

where

- rwnd is the receive window [bytes],
- RcvBuffer is the receive buffer [bytes],

- LastByteRcvd is the last byte in the receive buffer from the network layer [bytes],
- LastByteRead is the last byte read from the receive buffer by the application layer [bytes].

Informally, a measure of the spare room in the receive buffer.

3.7 TCP Unacknowledged Data Limit

$$\text{LastByteSent} - \text{LastByteAcked} \leq \min\{\text{cwnd}, \text{rwnd}\}$$

where

- LastByteSent is in [bytes],
- LastByteAcked is in [bytes],
- cwnd is the congestion window [bytes],
- rwnd is the receive window [bytes].

3.8 TCP Reno Average Throughput (Macroscopic Model)

$$\text{average throughput of a connection} = \frac{0.75 \cdot W}{\text{RTT}}$$

where

- average throughput of a connection is in [bits/sec],
- W is the maximum congestion window size, which occurs during a loss event [bytes],
- RTT is the round-trip time [sec].

3.9 TCP Average Rate with Loss

$$\approx \frac{1.22 \cdot \text{MSS}}{\text{RTT} \sqrt{L}}$$

where

- MSS is the maximum segment size [bytes],
- RTT is the round-trip time [sec],
- L is the loss rate.

4 Chapter 4

4.1 Buffer Size in Routers

Traditional wisdom suggests:

$$B = RTT \cdot C$$

However, more recent research suggests:

$$B = \frac{RTT \cdot C}{\sqrt{N}}$$

where

- RTT is the round-trip time [sec],
- C is the link capacity [bits/sec],
- N is the *independent* TCP flows sharing the link.

Based on more recent research, this formula applies when N is large such as in the network core.

4.2 Weighted Fair Queuing (WFQ)

In output port packet scheduling, each class i is guaranteed to receive a fraction of the service, given by

$$\frac{w_i}{\sum_{j=1}^N w_j}$$

where

- w_i is the weight of class i ,
- N is the number of classes with at least one packet in the queue.

5 Chapter 5

5.1 Link-State Algorithm Time Complexity

Without a heap to choose the next node with the least cost, the time complexity is

$$O(n^2)$$

where n is the number of nodes.

5.2 Bellman-Ford Algorithm

$$d_x(y) = \min_v \{c(x, v) + d_v(y)\}$$

$$D_x(y) = \min_v \{c(x, v) + D_v(y)\}$$

where

- $c(x, v)$ is the cost of the edge from node x to node v ,
- $d_x(y)$ is the shortest path from node x to node y ,
- $D_x(y)$ is the estimation of the least cost from node x to node y ,
- $d_v(y)$ is the shortest path from node v to node y ,
- $D_v(y)$ is the estimation of the least cost from node v to node y ,
- v is the set of neighbors of node x .

Eventually each $D_x(y)$ will converge to $d_x(y)$.

5.3 Link-State Message Complexity

The number of messages that need to be sent is

$$O(|N||E|),$$

where

- $|N|$ is the number of nodes (routers),
- $|E|$ is the number of edges (links).